

Detection of bike riding events with a mobile phone

SISTEMAS UBÍQUOS

Mestrado Integrado em Engenharia de Telecomunicações e Informática
Grupo P2:

- Manuel João Fernandes Mendes A77806
- Luís Pedro Bouças Araújo A81904
- Leandro Henrique Dantas Alves A82157
- José Eduardo da Silva Santos A82350
- Miguel Ângelo Teixeira Pereira A82875



Índice

Índice	2
Índice de figuras	3
Índice de tabelas.....	3
1 Motivação.....	4
2 Objetivos	4
3 Solução proposta	6
3.1 Arquitetura.....	6
3.1.1 Aplicação para treino do modelo	7
3.1.2 Sistema de modelação ML.....	8
3.1.3 Aplicação para deteção de eventos.....	8
3.1.4 Sistema de armazenamento de eventos	9
4 Avaliação	10
4.1 Algoritmo do modelo	10
4.2 Ajuste de eixos	13
4.3 Recolha de amostras para treino.....	15
4.3.1 Treino de buracos	15
4.3.2 Treino de travagens bruscas.....	22
4.3.3 Outros eventos	25
4.4 Aplicação para deteção de eventos.....	25
4.4.1 Deteção do modelo no dispositivo em tempo real	25
4.4.2 Paragens	29
4.4.3 Automatização do início e fim de percurso.....	30

4.5	Sistema de armazenamento	31
4.5.1	Página <i>web</i>	32
5	Conclusões	34
6	Trabalho futuro	36
7	Referências	38

Índice de figuras

Figura 3.1 – Arquitetura do sistema.....	6
Figura 4.1 – <i>Robust Scaler</i> aplicado a uma distribuição de valores. [12]	12
Figura 4.2 – Sistema de coordenadas do dispositivo. [13]	13
Figura 4.3 – Eixos não ajustados (a) e ajustados (b). [14]	14
Figura 4.4 – Depressão (buraco A) visualizada durante um percurso.	16
Figura 4.5 – Gráfico de aceleração linear na passagem de um buraco A.	17
Figura 4.6 – Gráfico de aceleração linear na passagem de um buraco B.	17
Figura 4.7 – Gráfico do giroscópio na passagem de um buraco A.	17
Figura 4.8 – Gráfico do giroscópio na passagem de um buraco B.	18
Figura 4.9 – Sistema de coordenadas do giroscópio.....	19
Figura 4.10 – Gráfico de aceleração linear na passagem por paralelos.....	19
Figura 4.11 – Gráfico de aceleração linear não filtrado na passagem de um buraco A.	20
Figura 4.12 – Matriz de confusão dos dois sensores.	20
Figura 4.13 – Matriz de confusão da aceleração linear.	21
Figura 4.14 – Gráfico de aceleração linear de uma travagem brusca A.....	22
Figura 4.15 – Gráfico de aceleração linear de uma travagem brusca B.....	23
Figura 4.16 – Gráfico de velocidade instantânea do GPS de uma travagem brusca A.	24
Figura 4.17 – Matriz de confusão dos buracos com travagens bruscas.....	25
Figura 4.18 – Matriz de confusão obtida pela camada <i>Batch Normalization</i>	26
Figura 4.19 – Detecção de buraco e travagem brusca.....	29
Figura 4.20 – Detecção de paragem.	30
Figura 4.21 – Página <i>web</i>	33
Figura 6.1 – Sistema de coordenadas do sensor geomagnético em relação à Terra. [1].....	37

Índice de tabelas

Tabela 4.1 Falsos negativos e positivos de cada combinação de eixos.	21
---	----

1 Motivação

Atualmente, o estado das estradas e caminhos para ciclistas é reconhecido normalmente apenas caso se tenha passado pelos mesmos previamente, e mesmo esse estado é muitas vezes temporário. Quando há certos obstáculos ou outras situações que devem ser previstas antecipadamente, o ciclista deve se precaver.

Tais obstáculos também devem ser do conhecimento da entidade governadora para que sejam tomadas medidas para os reduzir ou eliminar, se necessário.

2 Objetivos

Os objetivos deste projeto estão direcionados em encontrar uma forma de dar a conhecer, a ciclistas e câmaras municipais, a existência e localizações de determinados tipos de obstáculos.

Tais obstáculos devem providenciar informação útil para os motivos referidos. Portanto, idealmente estes seriam:

- **Depressões/buracos** – útil para informar os ciclistas acerca da existência de depressões/buracos para que a velocidade dos mesmos seja moderada antecipadamente e para informar a câmara municipal acerca da possível existência de mau estado do piso, sendo necessário ser reparado.
- **Travagens a fundo** – útil para indicar zonas onde há ocorrência frequente de travagens a fundo, que se podem dever à existência de obstáculos observados com pouco tempo de reação, possivelmente por estarem em zonas de pouca visibilidade, sendo necessária por parte dos ciclistas um aumento de atenção e moderação de velocidade antecipados.
- **Paragens (durante > 10 s)** – útil para indicar zonas onde o ciclista costuma parar e durante quanto tempo, que se podem referir, por exemplo, a estradas estreitas onde o ciclista deve ceder a vez a automóveis.
- **Lombas** – útil para informar a existência de lombas para que a velocidade seja moderada antecipadamente.
- **Subidas/descidas de passeio/degrau** – útil para informar a câmara municipal acerca das subidas de passeio mais frequentes que podem se tratar de zonas em que a rua é demasiado estreita para ser partilhada por automóveis e bicicletas ou, de zonas que necessitam de rampas caso sejam para entrar num estabelecimento.
- **Derrapagem** – útil para informar a existência de zonas de derrapagens frequentes para que a velocidade dos ciclistas seja moderada antecipadamente pois podem indicar humidade ou mau estado do piso.

Inicialmente, eventos como subidas, descidas, viragens à esquerda/direita, abrandamentos, acelerações e arranques rápidos também estavam incluídos nos planos, mas foram descartados por se tratarem de situações normais de condução, não trazendo qualquer tipo de informação útil para o pretendido.

Para haver recolha das situações de existência de eventos, das suas localizações geográficas e durações, é necessário um sistema com essa capacidade. Essa recolha deve ser feita por ciclistas comuns, pois são os que frequentemente testemunham os eventos. Sendo os ciclistas utilizadores comuns, estes não se devem preocupar em anotar manualmente a ocorrência de eventos, pois o foco deve se manter na condução, portanto deve ser uma recolha automatizada. Também pelo mesmo motivo, o instrumento de recolha deve ser um dispositivo possuído por todos os utilizadores.

A informação recolhida deve estar disponível numa plataforma de fácil acesso com dados claros para ciclistas e câmara municipal.

3 Solução proposta

3.1 Arquitetura

A arquitetura para a nossa solução proposta (Figura 3.1) inclui uma aplicação móvel por ciclista para recolha de amostras automatizada para deteção de eventos através dum modelo de *Machine Learning* (ML), uma página *web* com um mapa com a localização dos eventos ocorridos, um sistema para criação e treino do modelo de ML, e uma API (*Application Programming Interface*) com base de dados para envio, consulta e armazenamento de eventos.

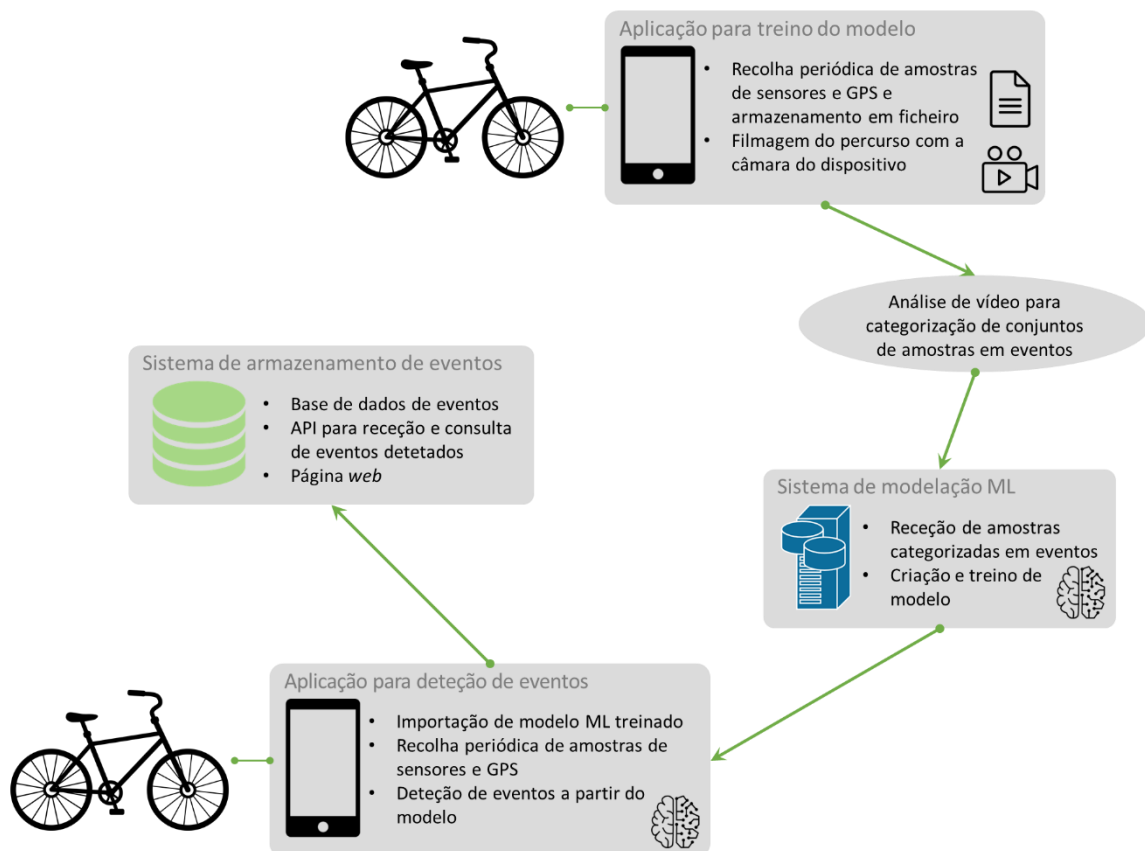


Figura 3.1 – Arquitetura do sistema.

3.1.1 Aplicação para treino do modelo

Através duma aplicação móvel para *smartphones* Android, o utilizador pode recolher amostras de dados de sensores a um intervalo de 10 ms.

Os tipos de dados considerados relevantes para os eventos pretendidos incluíram:

- Aceleração linear¹ [1] – aceleração em 3 eixos baseada no acelerómetro com efeito da gravidade removido
 - Eventos planeados: Todos, exceto paragem;
- Giroscópio [1] – velocidade angular em 3 eixos
 - Eventos planeados: Todos, exceto travagem a fundo e paragem;
- GPS (*Global Positioning System*) [2] – velocidade instantânea
 - Eventos planeados: Travagem a fundo.

Antes de ser dado o início à recolha, o utilizador deve ajustar a inclinação dos eixos para se adaptar à inclinação do dispositivo, que se deve encontrar num suporte fixo no guiador da bicicleta. Este ajuste deve ser feito num só clique num botão que, a partir do sensor de gravidade¹, roda os eixos de modo a ficarem com valores $(x, y, z) = (0,0; 9,80665; 0,0)$ m/s².

Em segundo lugar, deve ser ativado um gravador de ecrã, desde que a aplicação contenha uma prévisualização da câmara do dispositivo, de modo a ser filmado o percurso, e principalmente a roda dianteira para saber o momento em que invoca um evento. Para eventos “invisíveis” pela câmara, mas que o utilizador sabe quando ocorrem, quando são invocados, podem ser assinalados pela voz do utilizador no instante em que acontecem, sendo necessário que o gravador de ecrã tenha também acesso ao microfone.

Para um treino eficaz, devem ser recolhidos muitas amostras dos eventos pretendidos, assim como amostras cujos sensores podem confundir com eventos, quando na verdade podem se referir a uma característica normal do piso ou do movimento da bicicleta.

Durante o percurso, as amostras recolhidas devem ser armazenadas num ficheiro.

¹ *accelerometer = gravity + linear acceleration*

3.1.2 Sistema de modelação ML

Para treinar um modelo de ML, as amostras recolhidas devem ser agrupadas em ordem temporal em conjuntos categorizados com o tipo de evento ocorrido. A ocorrência do evento deve ser procurada e visualizada a partir da filmagem feita durante o percurso, podendo ser suportado por gráficos caso encontre semelhanças entre eventos da mesma categoria. A inexistência de eventos (*idle*) também deve ser considerada, sendo as suas amostras igualmente necessárias para a criação do modelo.

Após agrupadas as amostras em conjuntos categorizados por eventos (ou falta deles), devem ser usados como *input* do modelo.

Como as amostras de um conjunto seguem uma sequência temporal com uma frequência de amostragem fixa, este modelo deve ser do tipo temporal sequencial, treinado e testado a partir de uma janela deslizante.

Os conjuntos de amostras adquiridos também devem ser separados em dois grupos – treino (para construir o modelo) e teste (para determinar a sua eficácia).

O modelo construído é útil para eventos que manifestem padrões não lineares distintos e em múltiplos eixos, contudo, eventos como o de paragem, apenas necessitam de um valor médio da velocidade instantânea obtida pelo GPS num determinado intervalo de tempo para determinar se o ciclista se encontra parado.

3.1.3 Aplicação para deteção de eventos

A aplicação para deteção de eventos, à semelhança da aplicação de treino do modelo, deve recolher dados dos mesmos tipos de sensores e com a mesma frequência de amostragem. Adicionalmente deve também recolher:

- GPS – latitude e longitude – metadados de todos os eventos para poderem ser localizados no mapa
- *Timestamp* – metadados de todos os eventos para poderem ser localizados temporalmente

A deteção de eventos pode ser feita de duas formas – feita no dispositivo, ou feita num servidor.

A aplicação deve ser o mais automatizado possível, não devendo ser necessário que o ciclista assinale manualmente o início e término do seu percurso. Felizmente, dispositivos Android dispõem da Activity Recognition API [3] que permite detetar o estado do dispositivo, e os estados disponíveis incluem o “*On Bicycle*”, acionado quando o utilizador começa a andar de bicicleta, assim como quando termina.

Esta API deteta atividade ao ler periodicamente curtos conjuntos de dados de sensores e processando-os usando modelos de ML de uma forma que optimize os recursos dos dispositivos. É então ideal para invocar o início e o fim de percursos.

O ajuste de eixos também deve ser feito automaticamente e no momento do início do percurso.

Idealmente, nem devia ser necessário abrir a aplicação, pois a Activity Recognition API é capaz de correr em *background*, que podia abrir a aplicação automaticamente nas condições referidas e deixá-la correr em *foreground* (recolhas consistentes e, se for caso disso, possível deteção de eventos no dispositivo, por serem consideradas tarefas pesadas, não podem ser corridas em *background*).

3.1.4 Sistema de armazenamento de eventos

O sistema de armazenamento de eventos deverá despor de uma base de dados com uma lista de eventos com os seguintes componentes:

- Designação do evento;
- *Timestamp*;
- Coordenadas geográficas;
- Duração (exclusivo a paragens);
- Código identificador do percurso – atribuído aleatoriamente pelo dispositivo; útil, por exemplo para eliminar eventos de paragem detetados durante o término de um percurso.

Esta lista deve ser preenchida sempre que é detetado um evento, sendo o conhecimento deste sistema feito a partir de uma API acedida pela aplicação de deteção de eventos.

Esta API também deve ser acedida pela página *web*, responsável por apresentar as ocorrências de eventos num mapa.

4 Avaliação

4.1 Algoritmo do modelo

Durante o desenvolvimento deste projeto, o nosso desafio principal era em encontrar o algoritmo ideal para construir um bom modelo de *Machine Learning* para este sistema. A *framework* que nos despertou mais interesse foi o TensorFlow [4], pois era suportada por diversos tutoriais e achamos que, como nunca tínhamos trabalhado com ML anteriormente, seria a ideal para aprendermos.

Descobrimos então que um modelo de ML, como o nome indica, é construído à base de aprendizagem, sendo necessário ser ensinado. Quisemos então ensinar o modelo o que era um buraco – o nosso primeiro evento, e como a aprendizagem melhora com as experiências, quantos mais buracos ensinados, melhor.

Rapidamente reparamos que a “imagem” de um buraco teria de ser mostrada ao sistema em forma de amostras de sensores numa sequência temporal e que provavelmente acelerómetros e giroscópios seriam os ideais, pois são afetados por movimentos bruscos que é o que buracos causam. Também prevemos que todos os buracos geram padrões semelhantes pois todos os buracos têm características únicas que os definem como buracos.

Tentamos procurar tutoriais com este mesmo tema, mas foi difícil encontrar. Um modelo que achamos que seria semelhante foi o do reconhecimento de sons, pois também se tratam de padrões em forma de sequências temporais. Encontramos então um tutorial para construir um modelo de deteção de sons de morcegos [5]. Uma característica deste modelo que achamos estranha e que se assemelhava a outros tutoriais, como a deteção de gestos com luvas de boxe [6], foi o evento “*idle*” que também era categorizado com um tipo de evento. Achamos desnecessário considerar este tipo de evento pois, a nosso ver, modelos como deteção de objetos [7] apenas contém informação dos objetos e não da falta deles; contudo, não encontramos informação acerca deste facto neste tipo de modelo, portanto não pudemos confirmar nada. Tínhamos pensado num modelo que apenas detetasse padrões e que teria um *threshold* definido que definia o “quão parecido” um conjunto de amostras teria de ser. Tomamos então a decisão de optar por incluir eventos “*idle*” no nosso modelo. Outro tutorial do tema, neste caso de deteção de gestos em dispositivos Android [8], consistia em dois eventos (mover para a direita e mover para a esquerda), e em vez de treinar o modelo com amostras em *idle*, definiu um *threshold* baseado nas probabilidades de cada um dos modelos; se essas fossem próximas, não era considerado nada, se o *threshold* fosse ultrapassado pela probabilidade de um dos eventos, esse evento seria detetado. Decidimos não experimentar este modelo porque requeria pelo menos dois eventos, e queríamos um modelo capaz de suportar um só evento para o caso de não haver tempo para mais.

Também reparamos na maior parte dos tutoriais baseados em sensores de movimento de *smartphones* que, em vez de usarem acelerómetros, usavam o sensor de aceleração linear que é geralmente ideal para sistemas de *dead reckoning* [1] e que, como

previamente referido, exclui o efeito da gravidade, apresentando em repouso os valores (0,0; 0,0; 0,0) m/s².

Para contruirmos o nosso modelo, baseamo-nos num tutorial de reconhecimento de atividades de uma pessoa [9].

O modelo foi construído em Python através da API Keras [10] com o tipo temporal sequencial. Este tipo é caracterizado pela capacidade de empilhar camadas, sendo uma delas uma rede LSTM (*Long Short-Term Memory*) [11], que é capaz de aprender e memorizar longas sequências de dados de entrada, que não precisam de ser tratados.

Neste modelo, tanto o treino como a deteção, foram feitos a partir de uma janela deslizante, com um tamanho e *step* fixos.

Este modelo também é suportado por um método de normalização *Robust Scaler*, que provoca o efeito da Figura 4.1.

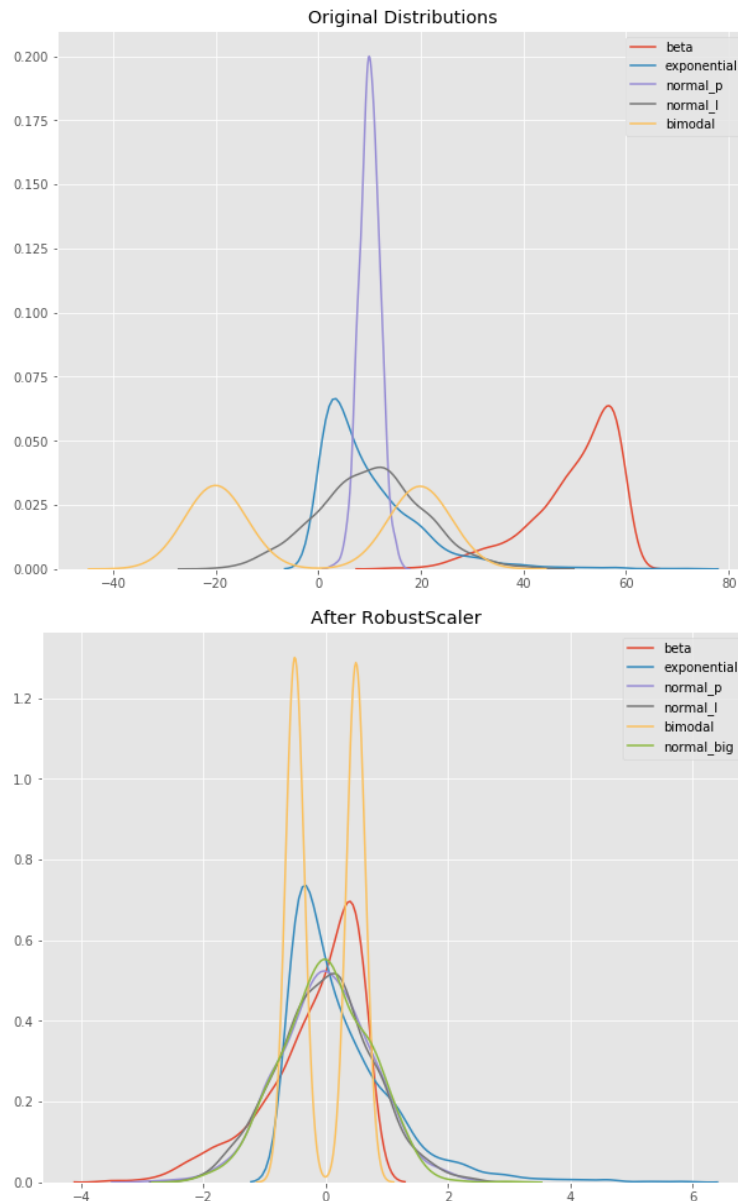


Figura 4.1 – *Robust Scaler* aplicado a uma distribuição de valores. [12]

Como o tamanho da janela é fixo, optamos também por um tamanho fixo do número de amostras por conjunto de um evento, igual para todos os tipos de eventos. Ao analisarmos o gráfico da passagem de um buraco (passagem pelas duas rodas) (Figura 4.5), reparamos que em média, a duração rondava os 0,8 segundos, que correspondia a 80 amostras a uma frequência de amostragem de 10 ms. De forma a abrangermos a totalidade da maior parte dos buracos, definimos o tamanho do conjunto como 100 (1 segundo). Sendo 100 também o tamanho da janela deslizante, com um *step* de 2 em 2 amostras. Amostras *idle* também foram agrupadas em conjuntos de 100.

Também foi definido o número de épocas de treino como 10. Quanto maior este número, mais preciso se torna o modelo, contudo, também se torna mais demorada a sua construção. E apesar da duração ser proporcional ao número de épocas, a precisão tem um melhoramento logarítmico, pelo que aumentar demasiado o número de épocas não compensaria a duração da sua construção.

4.2 Ajuste de eixos

Como referido anteriormente, é importante que os eixos de aceleração do dispositivo sejam ajustados de forma a que o utilizador não tenha de colocar o dispositivo numa inclinação específica em relação à bicicleta. Pretendeu-se então ajustar os eixos de forma a que, em vez dos eixos da Figura 4.2 usarem como referência o telemóvel, fosse usada a bicicleta como essa referência.

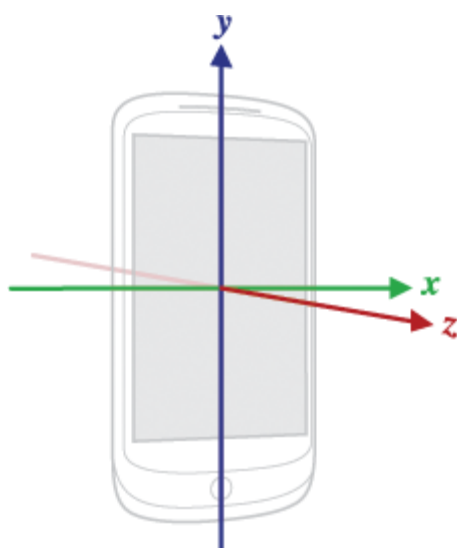


Figura 4.2 – Sistema de coordenadas do dispositivo. [13]

Nesta posição, os eixos, afetados pela aceleração gravítica, devem assumir os valores $(x, y, z) = (0, 0; 9,80665; 0, 0) \text{ m/s}^2$.

A inclinação é determinada pelo efeito que a aceleração gravítica tem sobre os três eixos de aceleração que, a partir dos valores recolhidos nesses eixos num instante enquanto o dispositivo se encontra fixo na bicicleta, consegue criar uma matriz de rotação R , usando as seguintes fórmulas [14]:

$X, Y, Z \rightarrow$ eixos de aceleração

$$\vec{V}_i = X\vec{i} + Y\vec{j} + Z\vec{k}$$

$$\alpha = \arccos\left(\frac{Y}{\|\vec{V}_i\|}\right)$$

$$\vec{A}_{norm} = -\left(\frac{Z}{\sqrt{X^2 + Z^2}}\right)\vec{i} + \left(\frac{X}{\sqrt{X^2 + Z^2}}\right)\vec{k}$$

$$q_0 = \cos(\alpha/2)$$

$$q_1 = \sin(\alpha/2) A_{norm,X}$$

$$q_2 = \sin(\alpha/2) A_{norm,Y}$$

$$q_3 = \sin(\alpha/2) A_{norm,Z}$$

$$R = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1q_2 - q_0q_3) & 2(q_0q_2 - q_1q_3) \\ 2(q_1q_2 - q_0q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_0q_1 - q_2q_3) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix}$$

Após ajustados os eixos, quando é recolhida uma amostra de qualquer tipo de sensor tridimensional, os três valores são rodados pela matriz de rotação a partir da seguinte fórmula:

$$\begin{Bmatrix} X' \\ Y' \\ Z' \end{Bmatrix} = R \begin{Bmatrix} X \\ Y \\ Z \end{Bmatrix}$$

É então feita a rotação demonstrada na Figura 4.3.

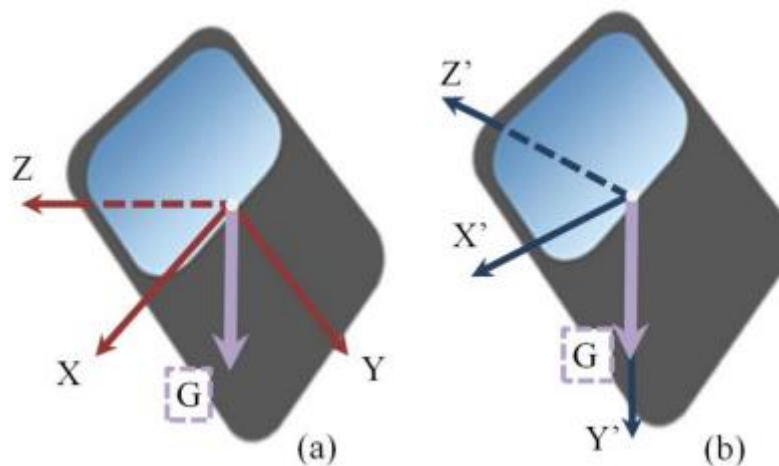


Figura 4.3 – Eixos não ajustados (a) e ajustados (b). [14]

Inicialmente, quando desconhecíamos os sensores de aceleração linear e de gravidade, o sensor escolhido para o ajuste foi o acelerómetro, que, por ser afetado por acelerações do movimento, requer que o dispositivo esteja parado (ou pelo menos em velocidade constante) para que a matriz de rotação seja calculada. Além disso, o ruído existente no sensor pode ser problemático.

Descobrimos então que o sensor de gravidade [1] seria a melhor opção, pois não é afetado por movimentos, apenas pela aceleração exercida pela gravidade, além disso o ruído é invisível (módulo dos eixos constantemente $9,80665 \pm 0,00001 \text{ m/s}^2$).

4.3 Recolha de amostras para treino

Desenvolvemos uma aplicação para dispositivos Android em Android Studio [15] com o propósito de recolher amostras num ritmo de 10 ms de dados recolhidos pelos 3 eixos do sensor de aceleração linear (atualizados a cada 5 ms), 3 eixos de giroscópio (atualizados a cada 5 ms) e de localização/velocidade do GPS (atualizadas a cada 1 segundo).

Durante a recolha, preocupámo-nos em recolher passagens pelos eventos pretendidos, e também passagens por situações que se podem assemelhar a eventos, mas que na verdade correspondem ao estado *idle*.

Para confirmar a compatibilidade da recolha em diversos dispositivos e respetivas inclinações, as amostras foram recolhidas em dois dispositivos diferentes.

4.3.1 Treino de buracos

O treino de buracos incluiu passagens por 56 buracos e por situações possivelmente confundíveis com buracos, nomeadamente passagem em piso de paralelos ou de formações que não correspondem a buracos, mas que provocam algumas oscilações.

Em algumas situações, nomeadamente em passagens por piso com mau estado, foi difícil saber o que considerar como um buraco, mesmo visualizando a gravação.

Apesar de tampas de saneamento com alguma profundidade e algumas depressões não significarem mau estado no piso, foram mais acessíveis para serem passadas durante a recolha de amostras, pois buracos causados pelo mau estado no piso foram incomuns o suficiente para dificultar a recolha. Deste modo, as tampas e depressões referidas foram consideradas como buracos a detetar.

Depois de recolhidos, revemos a filmagem do percurso para se descobrir quando ocorreram as passagens por buracos, como a depressão da Figura 4.4.



Figura 4.4 – Depressão (buraco A) visualizada durante um percurso.

Os gráficos das amostras da aceleração linear (Figura 4.5 e Figura 4.6) e do giroscópio (Figura 4.7 e Figura 4.8) foram então posicionados nos índices da ocorrência dos buracos e foram observados os padrões encontrados que se assemelhavam entre si.

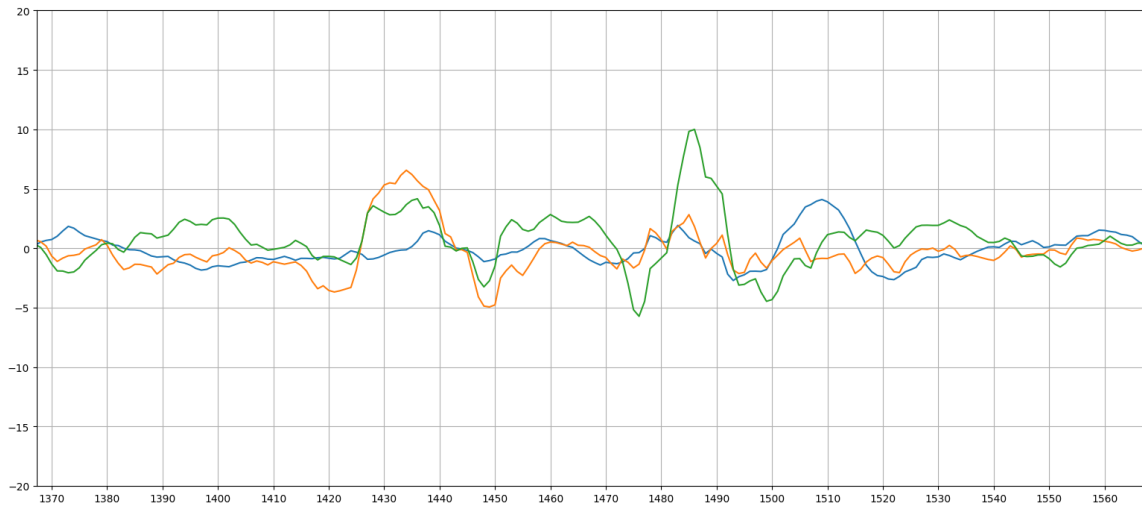


Figura 4.5 – Gráfico de aceleração linear na passagem de um buraco A.

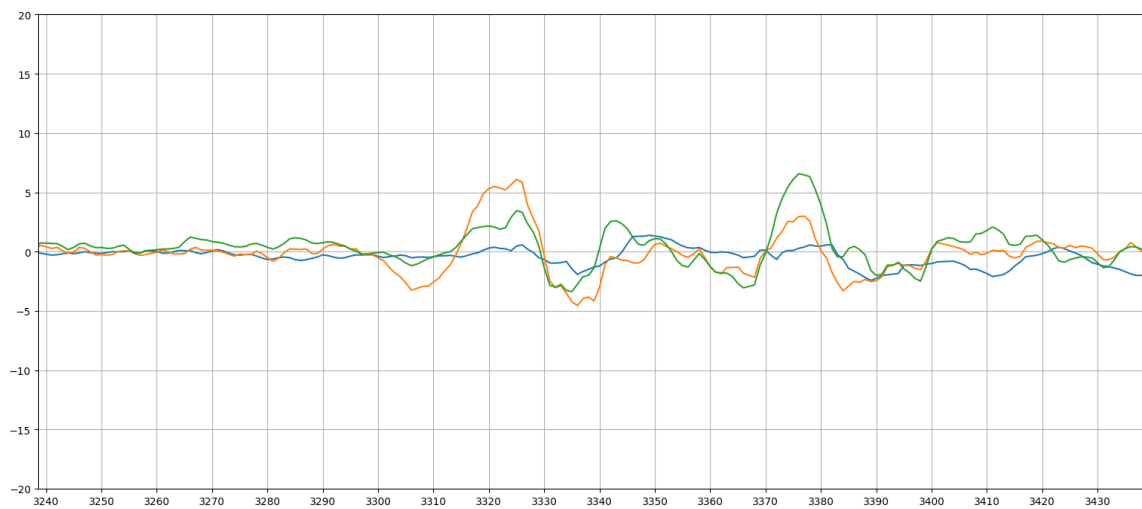


Figura 4.6 – Gráfico de aceleração linear na passagem de um buraco B.

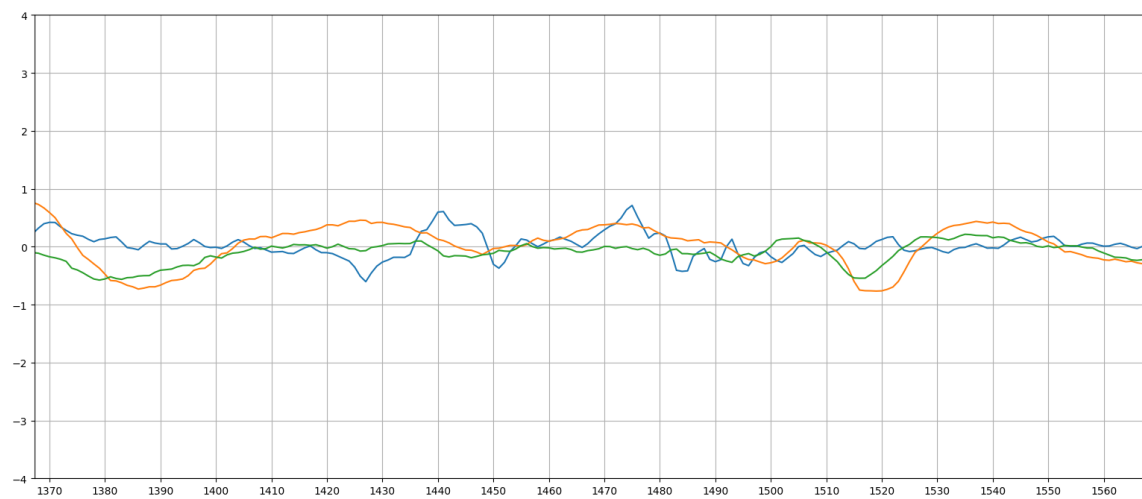


Figura 4.7 – Gráfico do giroscópio na passagem de um buraco A.

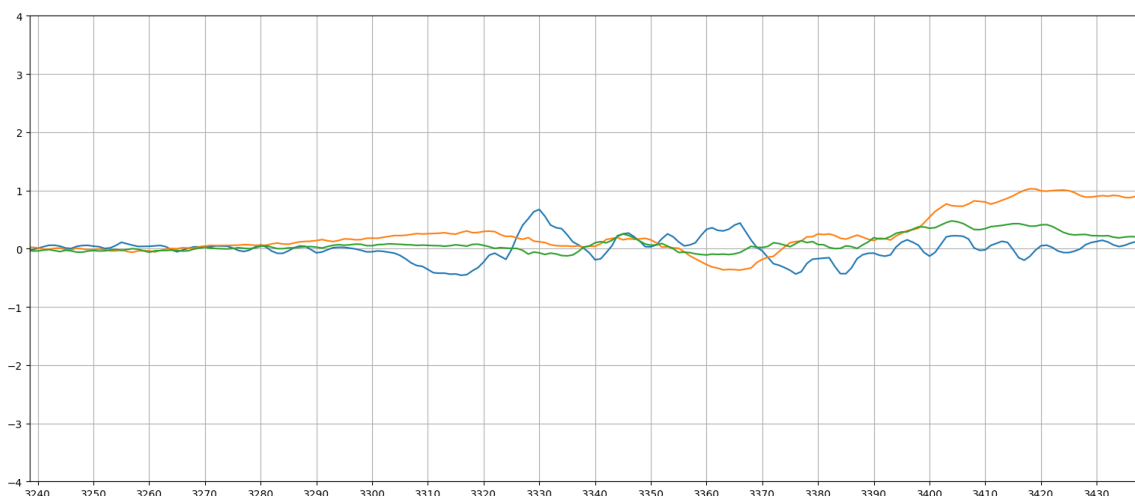


Figura 4.8 – Gráfico do giroscópio na passagem de um buraco B.

O eixo vertical dos gráficos da aceleração linear corresponde à aceleração em m/s^2 , o eixo vertical dos gráficos do giroscópio corresponde à velocidade angular em rad/s e o eixo horizontal em ambos os gráficos corresponde ao índice da amostra, que corresponde a 10 ms.

Como podemos verificar, há distinções entre duas das passagens em ambos os sensores.

Nestes dados podem ser descobertas as razões de terem estes valores, sabendo que a cor azul corresponde ao eixo X, o laranja ao eixo Y e o verde ao eixo Z e lembrando o posicionamento de cada eixo na Figura 4.2. No caso da aceleração linear:

- A existência de duas ondas de maior amplitude e comprimento corresponde à passagem das duas rodas da bicicleta.
- O eixo Y é o primeiro a ter os seus valores a decrescer, pois este eixo corresponde à direção da queda da roda dianteira no buraco – vertical. Na roda traseira, ocorre primeiro uma subida porque, como o dispositivo se encontra na parte da frente, há uma ligeira elevação.
- O eixo Z corresponde ao movimento em frente, que tem os seus valores aumentados pois ocorre uma travagem, e a travagem atua como uma aceleração que segue a direção do eixo.
- O eixo X é pouco afetado porque corresponde a movimentos laterais, que revelam pouco efeito nos buracos.

No caso do giroscópio, este tipo de sensor regista rotações nos mesmos três eixos da aceleração, representados na Figura 4.9.

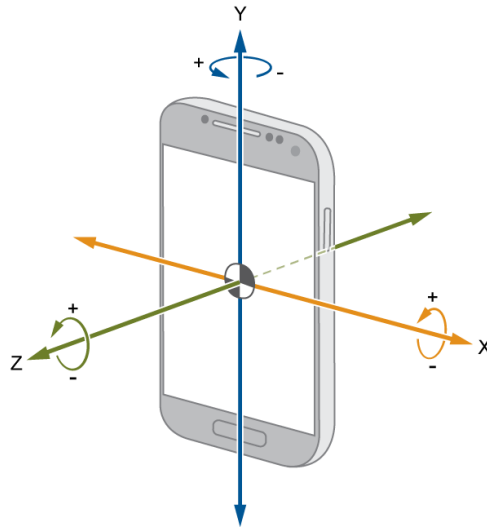


Figura 4.9 – Sistema de coordenadas do giroscópio.

Pode-se verificar que:

- A existência de duas ondas de maior amplitude e comprimento no eixo corresponde à passagem das duas rodas da bicicleta.
- O eixo X é o único com valores manifestantes pois a rotação ocorre em torno desse eixo, inicialmente negativos pois a roda da bicicleta toma uma rotação em torno dos ponteiros do relógio, ao contrário da rotação medida pelo eixo.

Como referido, uma das precauções consistiu em recolher amostras em paralelos para serem consideradas como *idle*, e como se pode verificar na Figura 4.10, apesar de o gráfico possuir ondas nos eixos Y e Z (da aceleração linear) à semelhança dos buracos, o número de ondas consecutivas e as reduzidas amplitude e comprimento de onda devem estabelecer a distinção.

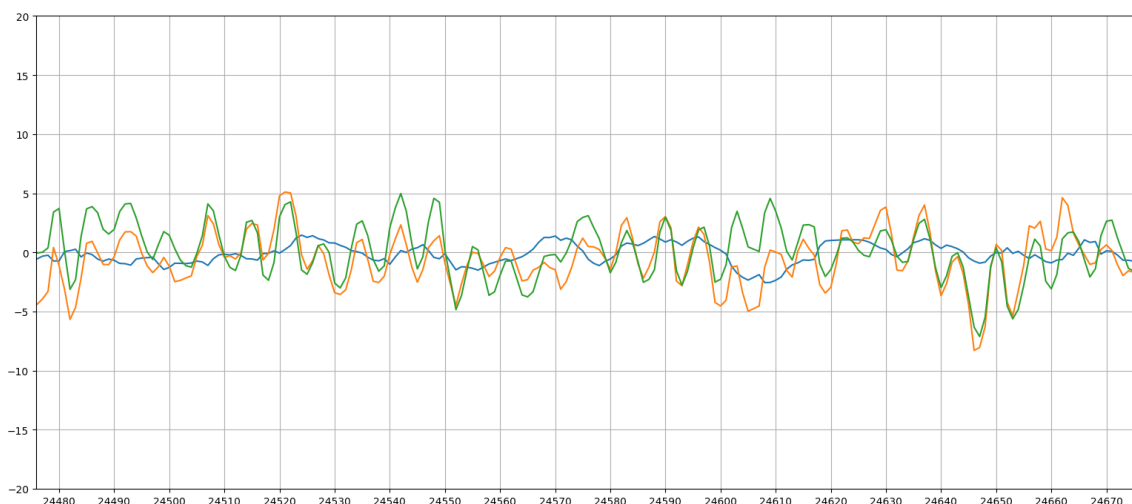


Figura 4.10 – Gráfico de aceleração linear na passagem por paralelos.

É de realçar que estes gráficos foram filtrados com um filtro passa-baixo para remover as altas frequências (Figura 4.11) possivelmente causadas por ruído nos sensores, possibilitando um tratamento mais claro dos dados.

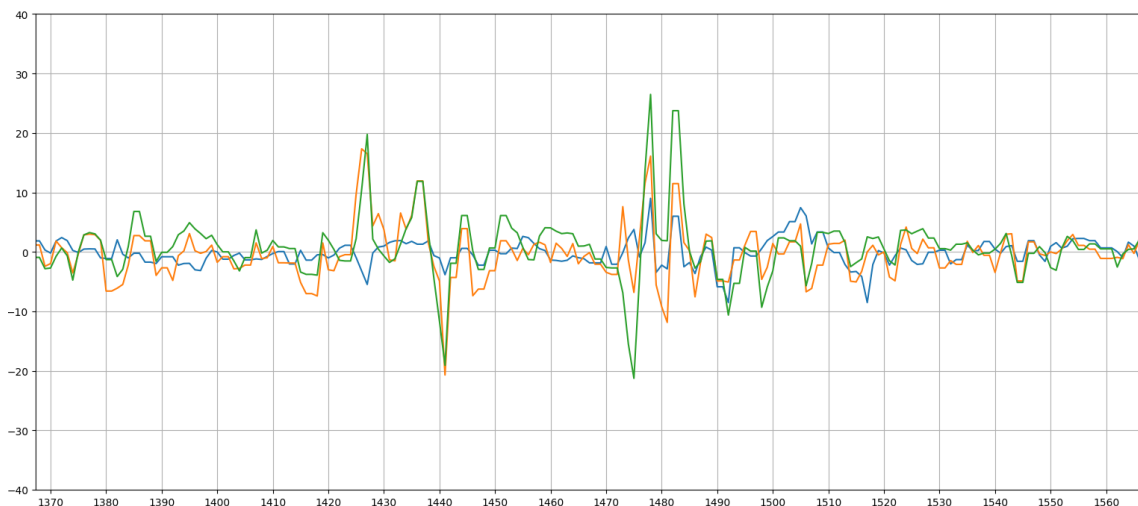


Figura 4.11 – Gráfico de aceleração linear não filtrado na passagem de um buraco A.

Treinamos então o modelo, multiplicando os valores do giroscópio por 4 para terem valores máximos semelhantes aos da aceleração linear, e separando os conjuntos em 40 conjuntos de buracos para o treino e 16 para o teste (os conjuntos *idle* foram divididos com a mesma proporção).

Na Figura 4.12 encontra-se a “matriz de confusão” correspondente aos resultados – falsos negativos e falsos positivos – alimentados pelos seis eixos que abrangem os dois sensores.

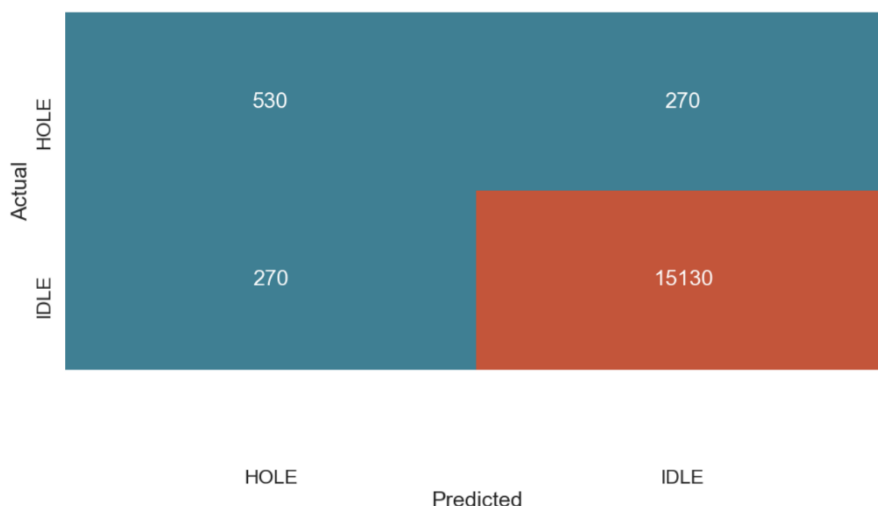


Figura 4.12 – Matriz de confusão dos dois sensores.

Os números representados correspondem aos resultados obtidos por cada janela percorrida² e pode-se comprovar na matriz uma percentagem de **33,75 %** falsos negativos e positivos (270 / (530 + 270)).

Optamos por remover um dos sensores, nomeadamente o giroscópio, para procurar melhorias no modelo, sendo obtida a matriz da Figura 4.13.

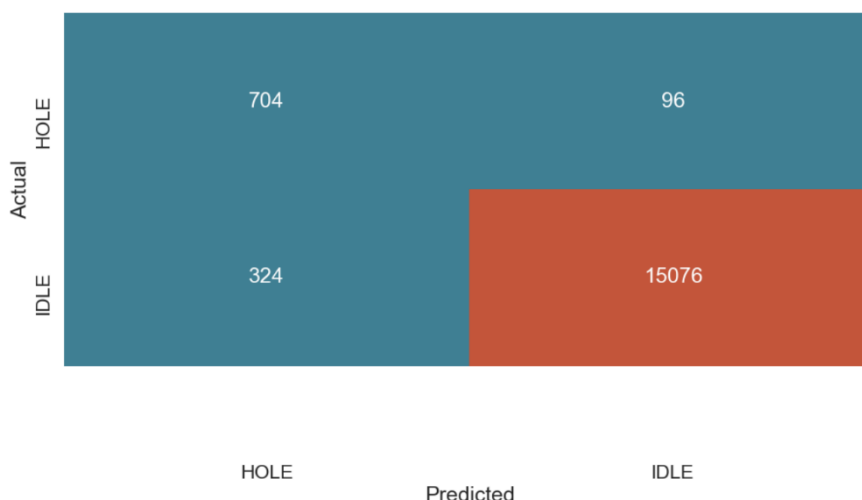


Figura 4.13 – Matriz de confusão da aceleração linear.

Nesta matriz verificou-se uma percentagem de **12 %** falsos negativos (96 / (704 + 96)) e **31,52 %** falsos positivos (324 / (704 + 324)). Devido a estes melhores resultados, optamos por descartar o giroscópio.

Para tentarmos melhorar os resultados, experimentamos aplicar os mesmos filtros usados nos gráficos, mas apenas pioraram, sendo então descartados.

Dentro dos valores da aceleração linear, testamos as diferentes combinações de eixos e reparamos na Tabela 4.1 que, apesar de algumas combinações de eixos mostrarem melhores resultados nos falsos positivos, a inclusão dos três eixos mostrou-se mais precisa no geral.

Tabela 4.1 Falsos negativos e positivos de cada combinação de eixos.

X	Y	Z	falsos negativos	falsos positivos
✓			54,78 %	31,67 %
	✓		29,04 %	30,83 %
		✓	20,11 %	28,09 %
✓	✓		32,72 %	27,67 %
✓		✓	9,07 %	34,51 %
	✓	✓	24,88 %	34,37 %
✓	✓	✓	12 %	31,52 %

² $N_{resultados} = \frac{L_{janela}}{L_{step}} \times N_{conjuntos} = \frac{100}{2} \times N_{conjuntos}$

Reparamos que a maior parte dos conjuntos de buracos se encontrava nas ocorrências onde realmente ocorreram buracos e em grupos densos. Para considerarmos a existência de um buraco, decidimos então agrupar os resultados em **conjuntos de 50**, e o evento ocorrido em maioria nesse conjunto é então considerado como o que realmente aconteceu.

4.3.2 Treino de travagens bruscas

Para as travagens bruscas, foi necessário algum cuidado, nomeadamente no que se deve considerar como travagem brusca, e como fazer com que o modelo não confunda uma travagem normal ou desaceleração com uma travagem brusca. Para este processo, foram recolhidas 35 amostras de travagens bruscas, enquanto também se faziam travagens não bruscas para serem consideradas como *idle*.

Como travagens são dificilmente visíveis numa filmagem (normalmente só é distinguível o momento em que se para), a recolha consistiu em usar o microfone para assinalar pela voz a ocorrência de travagens bruscas.

Na análise de dados, tínhamos já definido, pois parecia-nos óbvio, que o giroscópio não traria nenhuma utilidade, pois uma travagem geralmente não provoca rotações no dispositivo. Os melhores candidatos pareceram ser a aceleração linear e a velocidade instantânea do GPS, contudo este último precedia de algum receio devido a imprecisões e atrasos. Na Figura 4.14 e na Figura 4.15 encontram-se os gráficos de aceleração linear correspondentes a duas travagens bruscas.

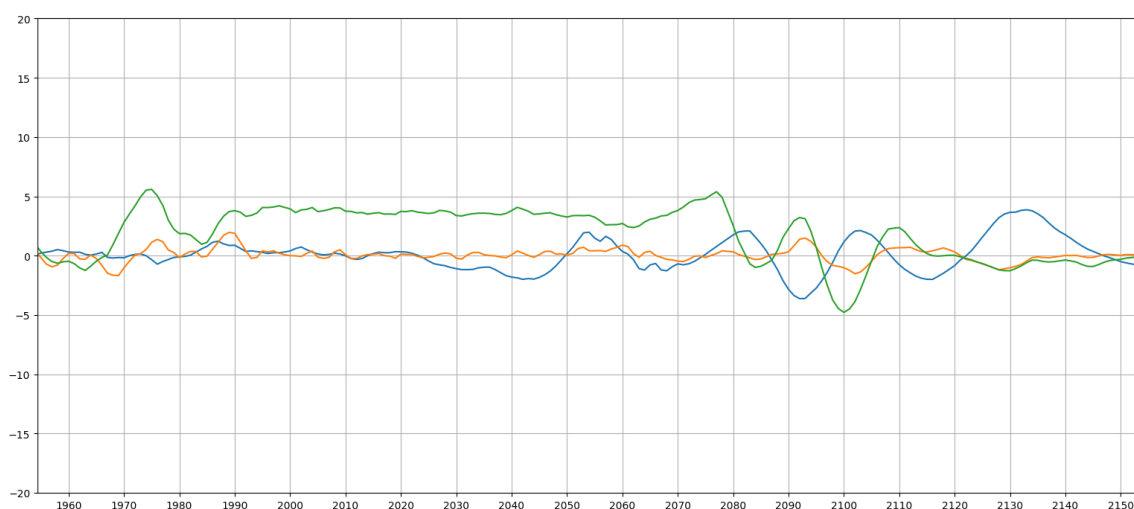


Figura 4.14 – Gráfico de aceleração linear de uma travagem brusca A.

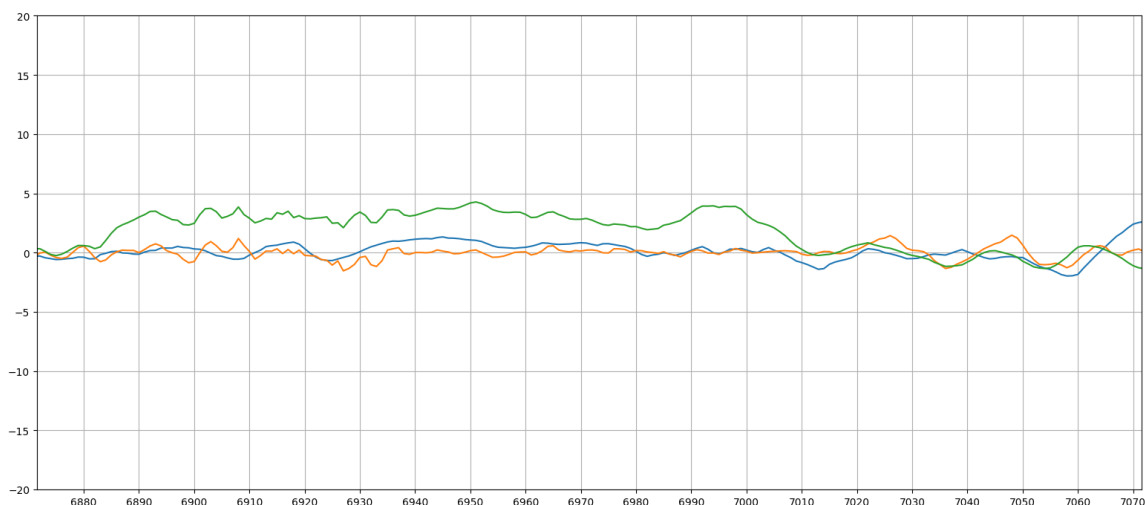


Figura 4.15 – Gráfico de aceleração linear de uma travagem brusca B.

Nestes dois gráficos, podem ser observadas semelhanças entre as duas travagens e os seus significados:

- O eixo Z foi o que mostrou um efeito considerável, geralmente linear, pois é o eixo afetado por acelerações no sentido contrário à orientação do movimento, portanto valores positivos correspondem a desacelerações, incluindo travagens. Travagens e desacelerações normais não possuem valores tão elevados neste eixo, pois não há tanta aceleração no eixo.
- Os gráficos terminam em valores a 0 que significam que a velocidade se encontra constante, ou uma paragem, que foi o caso.
- As oscilações nos eixos X e Z ocorridas no final da travagem podem se dever à falta de controlo da bicicleta causada pela inércia da força exercida ou a derrapagens em sentidos imprevistos, causando oscilações nesses eixos.

A velocidade instantânea do GPS mostrou-se inapta para ser usada neste tipo de evento, pois como se pode ver na Figura 4.16, apresentou descidas de velocidade um pouco atrasadas (3200 ms) e com baixa frequência de dados (período de 800 ms). O atraso até pode ser tolerável caso haja uma deslocação de fase dos valores; contudo, a baixa frequência pode ser demasiado lenta para confundir uma travagem brusca com uma lenta; além disso, zonas de pouca captação podem gerar valores inadequados.

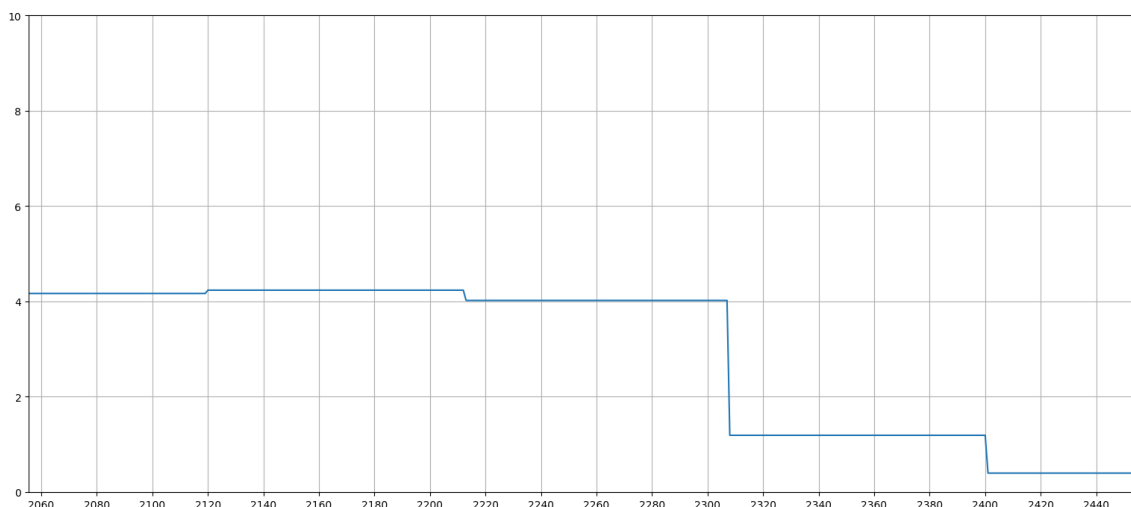


Figura 4.16 – Gráfico de velocidade instantânea do GPS de uma travagem brusca A.

Apesar de que, nos gráficos da aceleração linear, o eixo Z mostrar-se como o mais relevante, foram usados os três eixos no modelo, para o evento de travagem brusca possuir as mesmas variáveis que os restantes eventos.

Idealmente, um modelo devia ser capaz de ter um certo conjunto de variáveis específico, contudo, no que usamos, isto apenas seria possível:

- Se fosse criado um modelo para cada tipo de evento, tendo cada um as variáveis necessárias,
 - Seria um problema pois o componente de deteção teria de correr um modelo por tipo de evento e fazer um teste de deteção de cada conjunto de amostras por cada modelo existente;
- Se eixos a desconsiderar fossem na mesma usados, mas com valor 0,
 - Manteria o problema de fazer um teste de deteção de cada conjunto de amostras por cada modelo existente, em que cada teste incluiria os mesmos eixos, mas a 0 em determinados eventos.

Por este mesmo motivo, também foi mantido o tamanho de 100 amostras por conjunto, apesar de as travagens nos gráficos observados se manifestarem ao longo de 180 amostras. O conjunto de 100 foi então posicionado na zona mais icónica de uma travagem – a passagem da (des)aceleração constante para a falta de aceleração, que se trata do momento em que ocorreu a paragem. Por exemplo, na Figura 4.15, foi definido o intervalo [6950, 7050[.

Ao acrescentar este novo evento ao modelo previamente criado, foi obtida a matriz da Figura 4.17.

Actual	BREAK	412	10	128
	HOLE	0	651	149
	IDLE	121	320	32709
		BREAK	HOLE Predicted	IDLE

Figura 4.17 – Matriz de confusão dos buracos com travagens bruscas.

Nesta matriz verificamos as seguintes precisões:

- Nos buracos houve **18,63 %** falsos negativos $((0 + 149) / (0 + 651 + 149))$ e **33,64 %** falsos positivos $((10 + 320) / (10 + 651 + 320))$;
- Nas travagens bruscas houve **25,09 %** falsos negativos $((10 + 128) / (412 + 10 + 128))$ falsos negativos e **22,7 %** falsos positivos $((0 + 121) / (412 + 0 + 121))$.

A construção deste modelo demorou cerca de 150 segundos por época, totalizando aproximadamente **1500 segundos** (25 minutos).

4.3.3 Outros eventos

Apesar de terem estado nos nossos planos, não conseguimos treinar o modelo para mais eventos. Contudo, o evento de paragem foi um evento que arranjam forma de detetar, apesar de não ser via métodos de ML.

4.4 Aplicação para deteção de eventos

Para a deteção de eventos, foi aproveitada a aplicação de recolha para treino, adicionando um modo alternável de deteção.

A deteção de eventos assemelha-se ao treino na estrutura de *input* – são agrupadas 100 amostras num conjunto e esse conjunto é usado como teste, e o próximo conjunto corresponde ao anterior deslizado de modo a incluir a mais recente amostra recolhida, removendo a mais antiga, havendo igualmente uma janela deslizante.

4.4.1 Deteção do modelo no dispositivo em tempo real

No nosso planeamento inicial, a deteção dos eventos iria ser feita no sistema de armazenamento; contudo, por ser mais fácil de testar ao visualizar os alertas de eventos em tempo real, e porque era algo que nos enriqueceria mais o conhecimento nesta área comparando com manter a deteção no sistema, decidimos experimentar essa funcionalidade.

A importação e uso de modelos de TensorFlow são possibilitados pela sua *framework* de inferência *on-device* – TensorFlow Lite [16], ideal para dispositivos móveis e IoT (*Internet of Things*), com bibliotecas disponíveis para Android Studio.

Um dos desafios, foi adaptar os métodos de deteção existentes em Python para o ambiente Java do Android Studio. Infelizmente, não conseguimos encontrar formas de aplicar o *Robust Scaler*, mas arranhamos uma alternativa – substituir o *Robust Scaler* necessário no treino e no teste do modelo por um meio de normalização embebida no próprio modelo, não sendo necessário qualquer tipo de tratamento nos dados a testar. Este meio designa-se por *Batch Normalization* [17], que é uma camada de normalização do Keras aplicada após a aplicação da camada LSTM durante a construção do modelo, resultando na matriz da Figura 4.18.

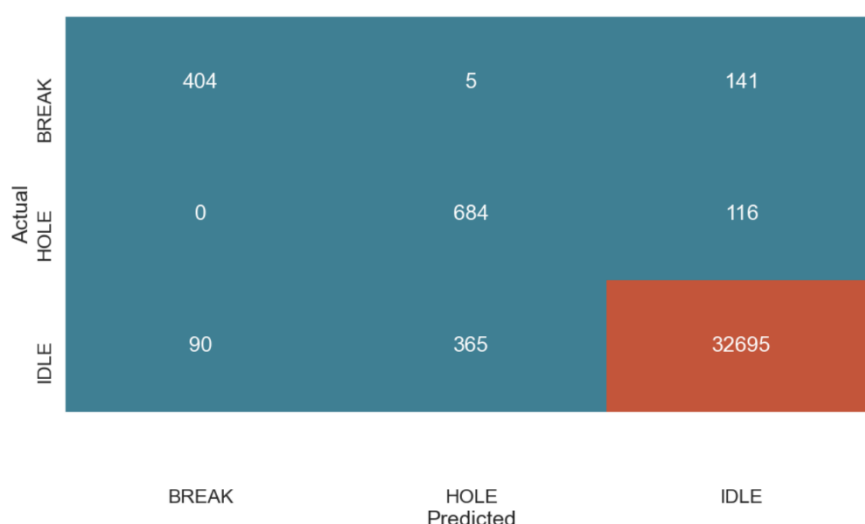


Figura 4.18 – Matriz de confusão obtida pela camada *Batch Normalization*.

Os resultados não foram muito afetados, sendo que:

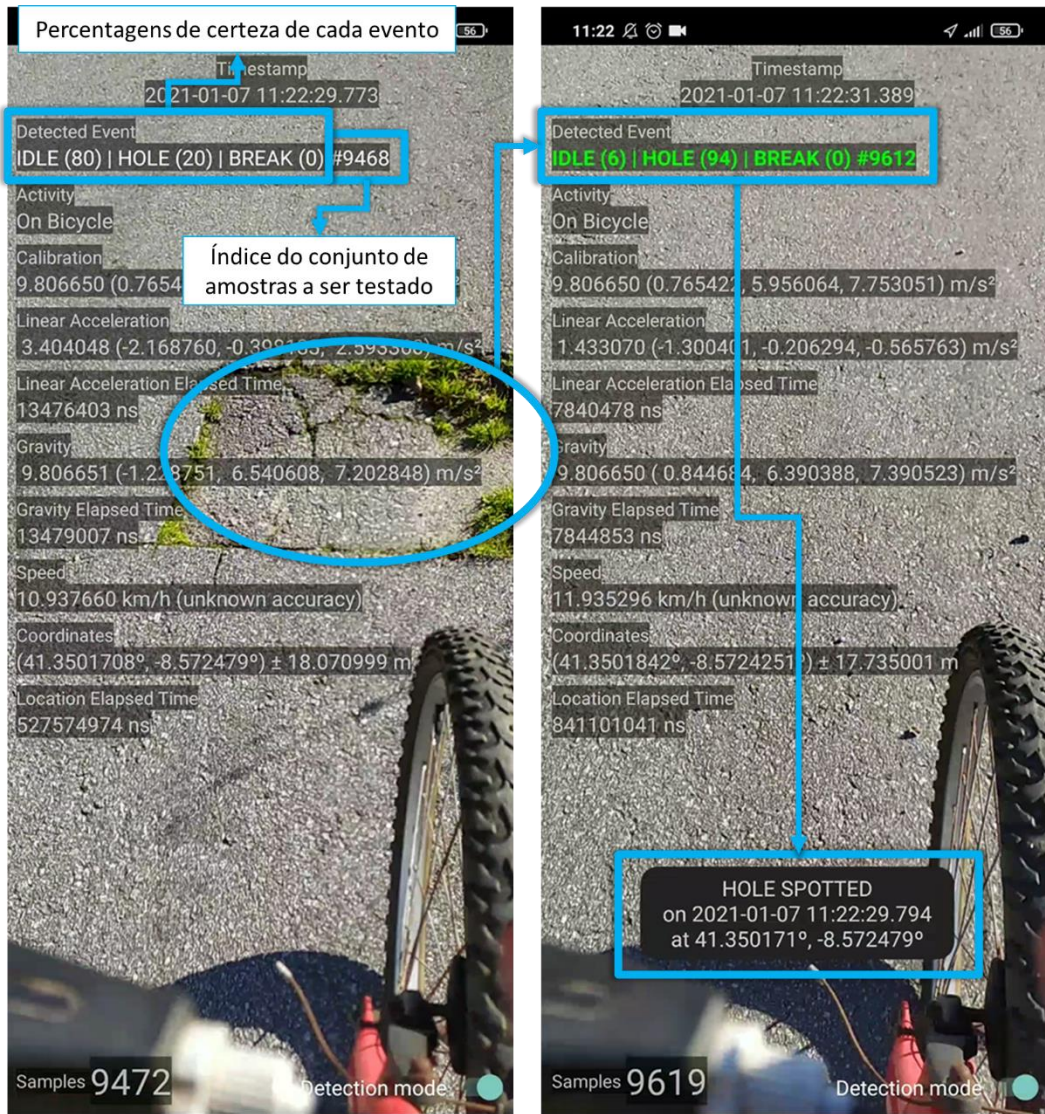
- Nos buracos houve **14,5 %** falsos negativos $((0 + 116) / (0 + 684 + 116))$ e **35,1 %** falsos positivos $((5 + 365) / (5 + 684 + 365))$;
- Nas travagens bruscas houve **26,55 %** falsos negativos $((5 + 141) / (404 + 5 + 141))$ falsos negativos e **18,22 %** falsos positivos $((0 + 90) / (404 + 0 + 90))$.

Após correr o modelo num telemóvel, verificou-se que o tempo de teste de um conjunto de amostras é superior à sua recolha, estando à volta do dobro (~20 ms). Como também referido, detetar em tempo real não é importante para este sistema, portanto foi implementado um *buffer* de amostras que é acrescentado sempre que são recolhidas e percorrido para serem feitos os testes. Isto continuava a não ser muito prático, tanto para testes em tempo real (para verificar se os alertas correspondiam a eventos reparados no momento) como no sentido de a aplicação ter de estar a correr depois do término do percurso para testar as amostras ainda não testadas.

Optou-se então por aumentar o *step* da janela para 2, sendo o teste feito a cada dois conjuntos. Esta opção até se mostrou útil, pois diminuiu o número de falsos positivos devido ao peso que os conjuntos de resultados de um evento tinham (é de lembrar que um evento é estabelecido pela maioria dos eventos detetados em 50 resultados consecutivos). Esta situação também deveria reduzir o número de eventos reais detetados, contudo, felizmente não houve grande diferença.

Apesar dos eventos serem detetados em tempo real, a comunicação com o sistema de armazenamento não é feita aquando da deteção. Os eventos detetados são armazenados no dispositivo e lançados todos numa só mensagem no fim do percurso. Isto é útil para descartar eventos de paragem detetados quando o percurso terminou, pois, o dispositivo terá esse evento armazenado antes de saber que o percurso terminou, e quando saber descartará esse evento. Além disso permite o envio via Wi-Fi, poupando dados móveis.

Na Figura 4.19 encontram-se três *snapshots* da aplicação desenvolvida.



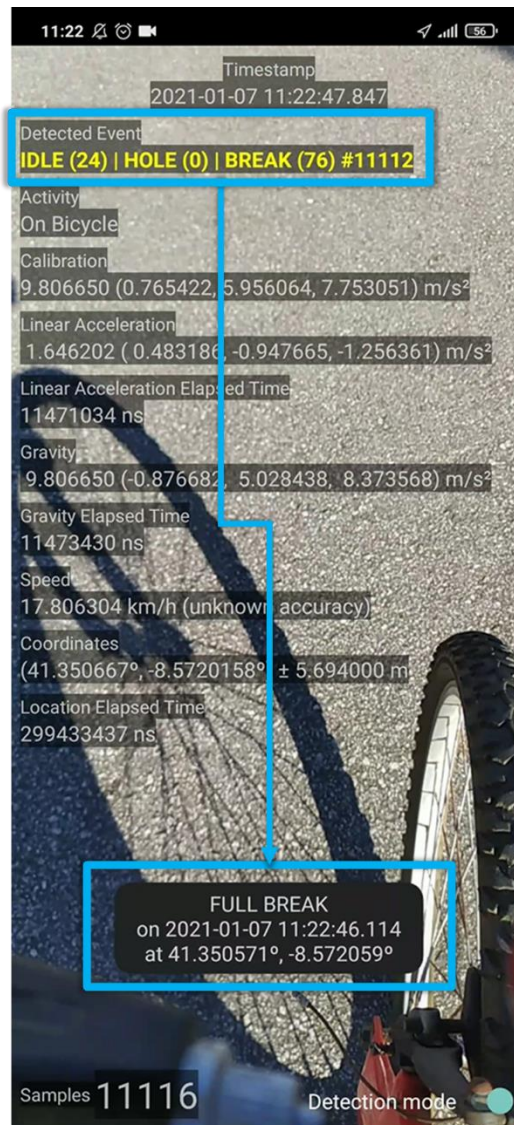


Figura 4.19 – Detecção de buraco e travagem brusca.

4.4.2 Paragens

O evento de paragens é detetado com base na velocidade instantânea do GPS do dispositivo.

Ao serem recolhidas 1000 amostras, correspondendo a 10 segundos, a média da velocidade contida em cada amostra determina se deve ou não ser anunciada a ocorrência de um evento de paragem. O evento de paragem é anunciado quando:

$$\tilde{s} \leq 1.8 \text{ (km/h)}$$

Foram escolhidos 10 segundos porque paragens com duração inferior a esse tempo normalmente correspondem a paragens sem nenhum motivo preocupante como a existência de obstáculos.

Durante a paragem, quando é recolhido um conjunto que termine essa condição, é calculada a duração da paragem pois é um parâmetro importante para o tipo de evento.

Se uma paragem for iniciada menos de 10 segundos após a anterior dar por terminada, a duração é acrescentada à anterior.

Na Figura 4.20 encontra-se um *snapshot* da deteção duma paragem.

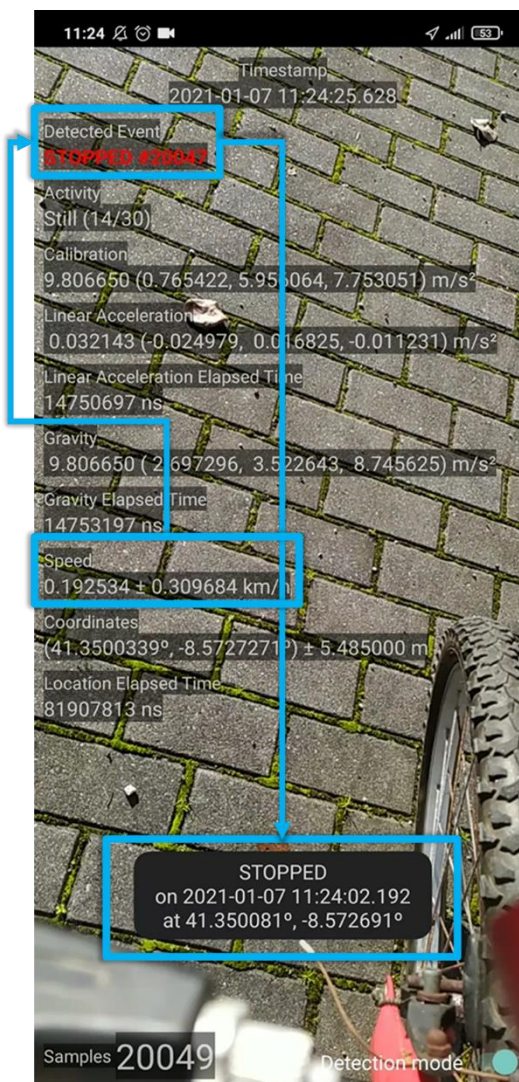


Figura 4.20 – Deteção de paragem.

4.4.3 Automatização do início e fim de percurso

A partir da Activity Recognition API, foi implementado um método de dar início e fim a um percurso. O início é dado pela transição para a atividade “*On Bicycle*” e o fim é dado após um *timeout* iniciado pela transição de fim da mesma atividade.

O ajuste de eixos também ocorre durante o início do percurso.

O *timeout* depende de um segundo tipo de atividade – a atividade “*Still*”, acionada pelo repouso do dispositivo. Esta atividade é importante para que, quando o ciclista se encontra parado ainda na bicicleta durante o seu percurso, não seja lançado o término do percurso, mas sim a ocorrência do evento de paragem. Quando há uma transição de fim de “*On Bicycle*”, é lançado um *timeout* de 10 segundos, apenas prolongado se durante esse tempo houver uma transição para “*Still*”, que também lançará um *timeout*, pois o utilizador pode ter abandonado a bicicleta e ter permanecido em repouso, sendo então este *timeout* de 30 segundos.

Quando o percurso é dado por terminado, a aplicação tem noção que não é nesse instante que o ciclista realmente terminou o seu percurso, mas sim antes ou pelo menos aquando do início da contagem do *timeout*. O instante do término do percurso é estabelecido pelo momento do lançamento do evento de paragem que ocorre durante a transição de fim da atividade “*On Bicycle*”, que dá início ao *timeout*. Esse evento é então descartado antes da lista dos eventos detetados ser submetida.

4.5 Sistema de armazenamento

Como meio de armazenar os eventos ocorridos em *cloud*, foi construído um sistema de armazenamento em *Firebase* [18]. Este sistema dispõe de uma base de dados em Cloud Firestore [19], sendo os dados armazenados para cada evento os referidos anteriormente:

- Identificador do tipo de evento;
- *Timestamp*;
- Coordenadas geográficas;
- Duração (exclusivo a paragens);
- Código identificador do percurso.

Esta base de dados é acedida por uma API desenvolvida em Node.js [20], disponibilizando *endpoints* de inserção e leitura via pedidos HTTP (*Hypertext Transfer Protocol*).

O *endpoint* para inserção de eventos é acedido pela aplicação de deteção de eventos e tem a seguinte estrutura:

HTTP POST /events:

- Corpo:
 - Identificador do percurso
 - Eventos detetados
 - Identificador do tipo de evento
 - *Timestamp*
 - Coordenadas geográficas
 - Duração da paragem (se não for paragem: -1)

O *endpoint* para a obtenção dos eventos é acedido pela página *web*, que são filtrados pela área do mapa visualizada, tendo então a seguinte estrutura:

HTTP GET /events/<parameters>:

- Parâmetros: coordenadas dos pontos a nordeste e sudeste do retângulo de visualização do mapa
- Resposta:
 - Marcadores:
 - Identificador do tipo de evento
 - Coordenadas geográficas
 - Número de eventos representados pelo marcador
 - Duração média da paragem (se não for paragem: -1)

Cada marcador indica a localização de um evento e representa todos os eventos ocorridos num raio de 5 metros desse primeiro evento (desde que sejam do mesmo tipo), sendo o número de eventos incluídos nesse raio revelado num campo de ocorrências. Num marcador de eventos de paragem, é revelada a duração média dos eventos ocorridos no raio desse marcador.

Estes marcadores tornam mais clara a visualização do mapa e marcam idealmente um só obstáculo, não uma passagem por um obstáculo. Além disso, um número de ocorrências elevado pode indicar uma urgência no tratamento desse obstáculo, enquanto que um número reduzido pode não significar muita importância ou significar pouca credibilidade, sendo por vezes um falso positivo.

4.5.1 Página *web*

Foi desenvolvida uma página *web*, também hospedada na mesma Firebase, com o propósito de apresentar, de forma clara, as ocorrências de eventos marcadas num mapa providenciado pela Google Maps JavaScript API [21].

Como referido, as ocorrências de eventos são obtidas pela API desenvolvida que devolve as coordenadas e outras informações dos marcadores a apresentar.

Os marcadores simbolizam o tipo de evento com uma cor, sendo o verde correspondente a buraco, amarelo a travagem brusca e vermelho a paragem.

Ao ser feito o clique num marcador, é revelado o número de ocorrências de eventos daquele tipo num raio de 5 metros do marcador. Se for um marcador de paragem, é adicionalmente revelada a duração média das paragens ocorridas, tendo em conta o número de ocorrências.

Um *snapshot* da página desenvolvida encontra-se na Figura 4.21.

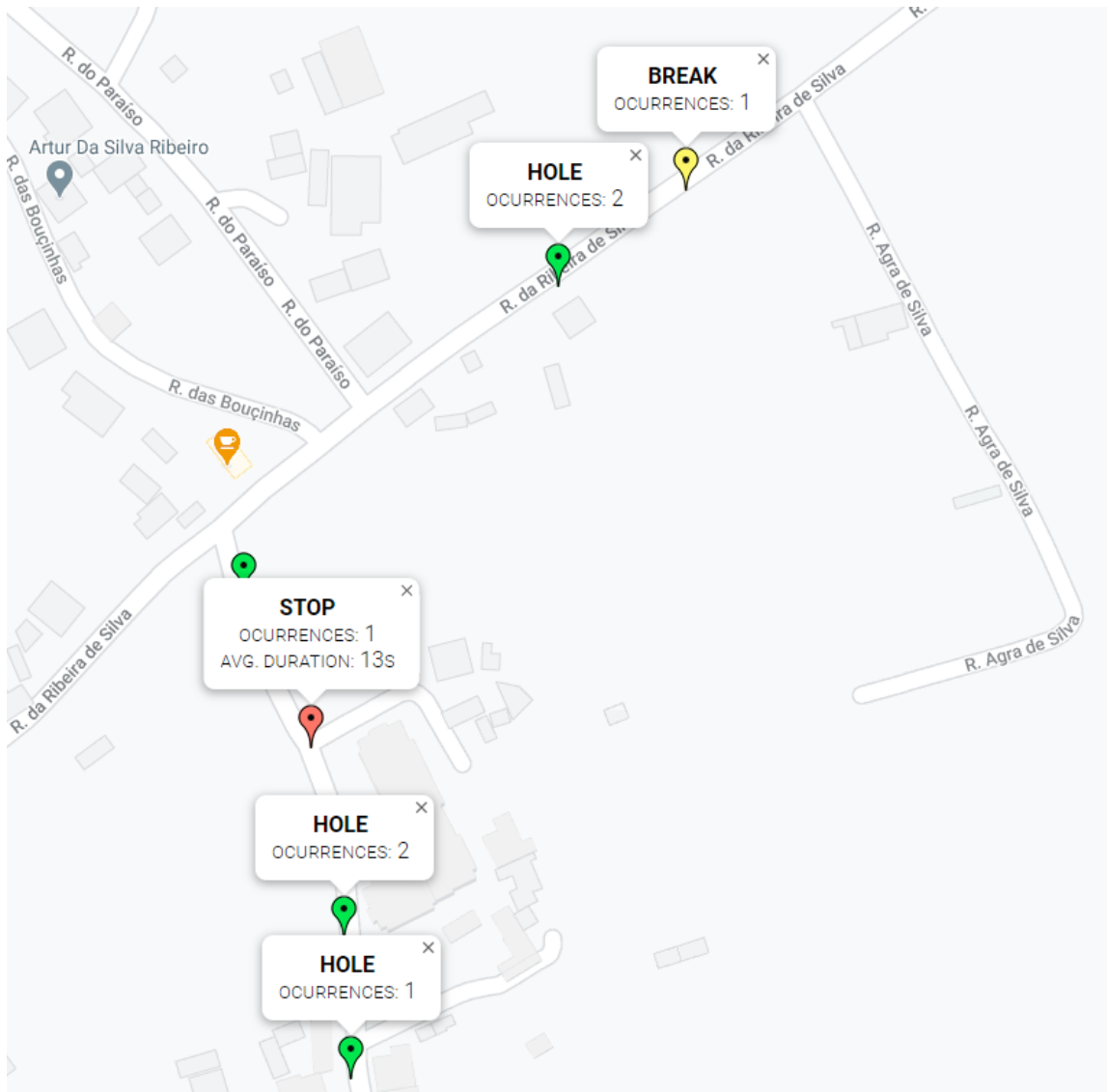


Figura 4.21 – Página web.

5 Conclusões

Neste projeto, consideramos que concluímos os principais objetivos propostos – construímos e treinamos um modelo com diferentes tipos de eventos detetáveis, aprendemos e colocamos em prática o conceito de *Machine Learning*, descobrindo o quão essencial se mostrou, também desenvolvemos uma aplicação de detecção direcionada a uma boa experiência de utilização, e um meio de visualização dos eventos detetados, para dar significado útil aos dados obtidos.

Contudo, reconhecemos que ficaram algumas imperfeições nas nossas implementações. Começando pela precisão da detecção de eventos, verificamos um número considerável de falsos negativos e falsos positivos existentes, e uma das razões é provavelmente o treino insuficiente do modelo, pois 56 buracos e 35 travagens bruscas são provavelmente números demasiado baixos. Outro problema foi em definir o *ground truth* durante o treino e testes, pois, no caso dos buracos, as características aceitáveis para definir um buraco são um pouco ambíguas e por vezes há buracos que não são notados pela gravação, causando algum engano no treino; isto também se aplica às travagens bruscas, pois não há um *threshold* globalmente definido para o quão desacelerado e a quanta velocidade inicial uma travagem deve ter para ser considerada brusca; quando estes problemas geram dúvidas no treino, é de esperar que as detecções também sejam por vezes duvidosas.

Provavelmente o problema que mais nos preocupou, visto que se refere ao objetivo principal deste projeto, foi de o tipo de modelo desenvolvido não parecer 100 % ideal para o pretendido, pois requer um número fixo de amostras por conjunto, e um número fixo de variáveis/eixos, dificultando a atribuição de tipos de eventos aos sensores que mostram mais relevância. Infelizmente não encontramos nenhuma solução a este problema para além do referido uso de um modelo para cada tipo de evento, que iria tornar o tempo de treino e detecção proporcional ao número de eventos, e seria um peso enorme para carregar, principalmente numa detecção *on-device* como a desenvolvida. Contudo, vários modelos não resolveriam o problema de tamanhos de amostras variados por tipo de amostra, sendo também necessário que a detecção e teste ocorram em tamanhos de janela variados em janela deslizante.

Durante a decisão de implementar a detecção de eventos no dispositivo, tínhamos a noção que teriam algumas desvantagens comparando com a detecção num servidor.

Deteção no dispositivo:

- Vantagens:
 - Dependendo das características do telemóvel, a deteção é feita em tempo real;
 - A comunicação com a API só é feita para enviar ocorrências de eventos e respetivas coordenadas geográficas e *timestamps*, havendo pouco consumo de dados móveis;
 - Mais privacidade do utilizador, pois o sistema de armazenamento apenas terá conhecimento das coordenadas geográficas dos eventos ocorridos, em vez de todo o percurso percorrido.
- Desvantagens:
 - Correr um modelo que teste amostras a cada 10 ms consome muita bateria;
 - Na maior parte dos dispositivos atuais, o teste de um conjunto de amostras pode chegar a demorar mais do dobro da duração da sua recolha (que se verificou em dois dispositivos), o que requer o funcionamento da aplicação após o percurso para conseguir testar todas os conjuntos recolhidos,
 - Uma forma de atenuar este problema foi reduzir a frequência de teste, sendo alguns conjuntos de amostras ignorados, que por vezes são redundantes.

Deteção no servidor:

- Vantagens:
 - O peso da deteção de eventos deixa de ser suportado pelo dispositivo, sendo poupada a bateria;
- Desvantagens:
 - Menos privacidade do utilizador, pois todos os dados GPS gerados durante o percurso são enviados;
 - Para a deteção ser feita em tempo real, as amostras devem ser constantemente enviadas durante o percurso, o que pode gerar um consumo elevado de dados móveis,
 - Contudo, este sistema não requer que os eventos sejam registados em tempo real, pois são sempre identificados com a *timestamp* em que ocorreram. Há então a alternativa de, após o término do percurso e após ser detetada uma conexão com Wi-Fi, ser submetido o ficheiro com as amostras recolhidas.

Quanto à importância dos falsos positivos e negativos, apesar de ser importante minimizar ambos, este sistema beneficia mais se a prioridade for reduzir ao máximo os falsos positivos. A razão para tal é de, na detecção de eventos, a detecção ser feita em horários variados e imprevisíveis e em passagens eventuais pelos eventos pretendidos, além disso quando há comunicação com o sistema de armazenamento, a informação enviada corresponde apenas à existência de eventos num determinado sítio; eventos *idle* nunca são submetidos, portanto **não é dito ao sistema onde é que não há eventos**, portanto quando uma zona do mapa não contém uma sinalização de um evento, não significa que não exista, apenas que **ainda** não foi detetado. Ou seja, mesmo que haja um falso negativo num determinado sítio, haverá sempre oportunidade desse evento ser posteriormente detetado por outro ciclista, ou pelo mesmo num instante diferente. Os eventos que se pretende detetar, apesar de incluírem obstáculos, não correspondem a situações de emergência.

6 Trabalho futuro

É difícil um projeto abordar todos os problemas envolvidos na solução de um problema, e este não foi exceção.

Durante a análise das amostras de treino, reparamos que a filmagem de buracos apenas revelava buracos passados pela roda dianteira, fazendo com que consultássemos o gráfico para descobrir a passagem da roda traseira, e também foi a razão de considerarmos um buraco como uma passagem de duas rodas e não de uma, pois se apenas a roda traseira passar por um buraco, dificilmente será descoberta essa ocorrência durante a análise. Além disso, estando o sensor apenas ao nível da roda dianteira faz com que os valores tenham algumas diferenças entre as passagens de ambas as rodas, como verificado anteriormente. Estes problemas seriam solucionados pela implementação de sensores e câmaras não só ao nível da roda dianteira, mas também da roda traseira.

Outro cuidado que se deve ter na recolha de amostras para treino é na adição de novos tipos de eventos, pois a passagem pelos mesmos pode ter sido previamente feita durante um treino e ter sido considerada como *idle*, provocando algum engano no modelo.

Quanto ao ajuste de eixos desenvolvido, a matriz de rotação tem de ser construída idealmente em plano ou pelo menos deve ser conhecida a inclinação do piso. Pois os eixos encontram-se referenciados no centro da Terra, não na bicicleta. Há três formas que descobrimos que poderiam ser usadas para calcular a inclinação do piso:

- Diferenças de altitude num intervalo de tempo obtidas pelo GPS,
 - Pode ser um problema devido a precisão insuficiente;
- Diferenças de altitude num intervalo de tempo obtidas pela Google Maps Elevation API [22],
 - Esta API disponibiliza as altitudes das superfícies, dadas as coordenadas geográficas, o que pode ser vantajoso por se tratarem de valores estáticos, o que não há muito que os influencie; além disso, é esperado que um ciclista esteja normalmente com as rodas no chão;
- Diferenças de altitude num intervalo de tempo obtidas por sensores de pressão atmosférica (barómetros/altímetros) [23],
 - A pressão atmosférica contém dados que podem ser usados para determinar altitude, mas pode ser afetada por condições meteorológicas; não foi testado porque os nossos dispositivos não se encontravam equipados com este sensor.

A orientação do dispositivo em torno do eixo Y do sensor de gravidade (perpendicular ao chão) também teve de ser específica, pois o este sensor é incapaz de se manifestar com tal orientação. Uma forma de corrigir este problema seria o uso de uma bússola ou sensor geomagnético (magnetómetro) [24] para determinar a orientação em relação ao norte magnético (determinada por Y (Figura 6.1)).

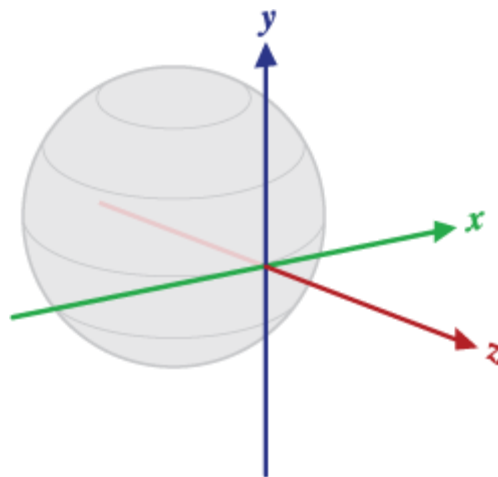


Figura 6.1 – Sistema de coordenadas do sensor geomagnético em relação à Terra. [1]

As aplicações móveis foram desenvolvidas em Android, mas beneficiarão da compatibilidade com iOS.

Apesar dos melhoramentos que devem ser feitos, este projeto também tem potencial para ser expandido pela adição de novas funcionalidades, como por exemplo integrar o mapeamento de obstáculos num sistema de navegação para bicicleta com alertas de voz na aproximação dos mesmos.

7 Referências

- [1] "Motion Sensors | Android Developers." https://developer.android.com/guide/topics/sensors/sensors_motion.html#sensors-motion-accel%0Ahttps://developer.android.com/guide/topics/sensors/sensors_motion.html (accessed Oct. 20, 2020).
- [2] "Location | Android Developers." <https://developer.android.com/reference/android/location/Location> (accessed Oct. 20, 2020).
- [3] "Activity Recognition API | Google Developers." <https://developers.google.com/location-context/activity-recognition/> (accessed Nov. 27, 2020).
- [4] "TensorFlow." <https://www.tensorflow.org/> (accessed Oct. 23, 2020).
- [5] R. Meertens, "Detecting bats by recognising their sound with Tensorflow - Pinch of Intelligence," *Pinch of Intelligence*, 2017. <http://www.pinchofintelligence.com/detecting-bats-recognising-sound-tensorflow/> (accessed Nov. 28, 2020).
- [6] "Boxing Punch Gesture Recognition — SensiML Documentation." <https://sensiml.com/documentation/application-tutorials/gesture-recognition-boxing-punches.html> (accessed Dec. 01, 2020).
- [7] "tensorflow Lite object detection." https://www.tensorflow.org/lite/models/object_detection/overview?hl=en (accessed Nov. 07, 2020).
- [8] R. Yanchyshyn, "Motion Gesture Detection Using Tensorflow on Android - Lemberg Solutions," *Lemberg Solutions*, 2017. <https://lembergsolutions.com/blog/motion-gesture-detection-using-tensorflow-android> (accessed Dec. 03, 2020).
- [9] V. Valkov, "Time Series Classification for Human Activity Recognition with LSTMs using TensorFlow 2 and Keras | by Venelin Valkov | Towards Data Science," *Towards Data Science*, 2020. <https://towardsdatascience.com/time-series-classification-for-human-activity-recognition-with-lstms-using-tensorflow-2-and-keras-b816431afdff> (accessed Dec. 11, 2020).
- [10] "Keras: the Python deep learning API." <https://keras.io/> (accessed Dec. 12, 2020).
- [11] "LSTM layer," *Keras*. <https://de.mathworks.com/help/deeplearning/ref/nnet.cnn.layer.lstm.html> (accessed Dec. 12, 2020).
- [12] J. Hale, "Scale, Standardize, or Normalize with Scikit-Learn – Towards Data Science," *Towards Data Science*. 2018, Accessed: Jan. 11, 2021. [Online]. Available: <https://towardsdatascience.com/scale-standardize-or-normalize-with-scikit->

learn-6ccc7d176a02.

- [13] "Sensor types | Android Open Source Project." <https://source.android.com/devices/sensors/sensor-types> (accessed Jan. 11, 2021).
- [14] M. D. Tundo, E. Lemaire, and N. Baddour, "Correcting Smartphone orientation for accelerometer-based analysis," *MeMeA 2013 - IEEE Int. Symp. Med. Meas. Appl. Proc.*, pp. 58–62, 2013, doi: 10.1109/MeMeA.2013.6549706.
- [15] "Download Android Studio and SDK Tools | Android Developers." <https://developer.android.com/studio> (accessed Oct. 17, 2020).
- [16] "TensorFlow Lite | ML for Mobile and Edge Devices." <https://www.tensorflow.org/lite> (accessed Dec. 09, 2021).
- [17] "BatchNormalization layer," *Keras*. https://keras.io/api/layers/normalization_layers/batch_normalization/ (accessed Dec. 29, 2021).
- [18] "Firebase." <https://firebase.google.com/> (accessed Oct. 30, 2020).
- [19] "Cloud Firestore Firebase." <https://firebase.google.com/docs/firestore> (accessed Jan. 06, 2021).
- [20] "Node.js." <https://nodejs.org/en/> (accessed Jan. 06, 2021).
- [21] "Overview | Maps JavaScript API | Google Developers." <https://developers.google.com/maps/documentation/javascript/overview> (accessed Jan. 07, 2021).
- [22] "Get Started | Elevation API | Google Developers." <https://developers.google.com/maps/documentation/elevation/start> (accessed Dec. 10, 2020).
- [23] "Environment Sensors | Android Developers." https://developer.android.com/guide/topics/sensors/sensors_environment (accessed Dec. 11, 2020).
- [24] "Position Sensors | Android Developers." https://developer.android.com/guide/topics/sensors/sensors_position (accessed Dec. 30, 2020).